

Figure 2: Experimental Results on the Solving Rate of METHODREFINE with Different Domain Incompleteness and HTN-MAKER

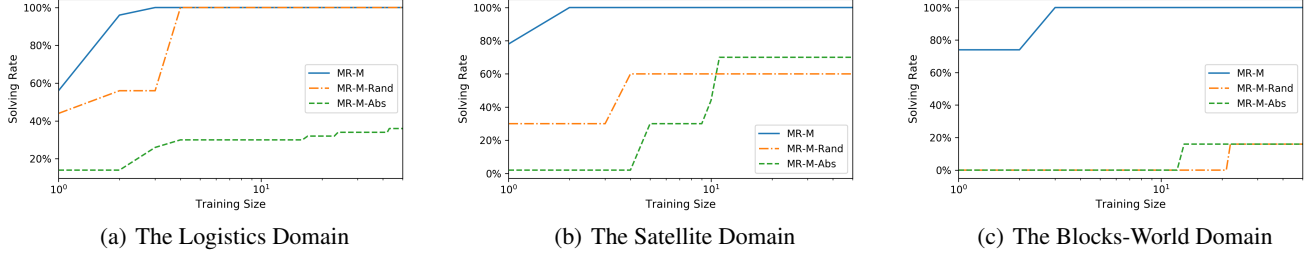


Figure 3: Experimental Results on the Solving Rate of METHODREFINE with Different Preferences

main but inferior on the Logistics and Satellite domain. In the Satellite domain, after training a number of instances, HTN-MAKER exceeds memory limitation when solving some instances, causing a sudden drop in its curve. It is because HTN-MAKER learns a method which causes an infinitely recursive decomposition. It never occurs in our approach because we only learn methods from acyclic decomposition trees. But they are solved when suitable methods are learned with the training set growing. HTN-MAKER finally learns 87, 23, 92 methods in the Logistics, Satellite, Blocks-World domain, respectively. Comparing with our approach, it learns more methods but many of which are redundant.

Experimental results on the performances of METHODREFINE with different preferences. To evaluate our assumption on the stratum-based prioritized preference, we also compare it against its opposite prioritization where tasks are inserted in as abstract compound tasks as possible ('MR-M-Abs'), and the case with no preference where any completion profile is allowed ('MR-M-Rand'). We choose the domains with the middle completeness.

From the experimental results (Figure 3), our stratum-based prioritized preference outperforms the other two cases. It is because the completion profile associates the inserted tasks to a more abstract task and it generates a more case-specific method which may not suit other instances.

Discussion and Conclusion

Without declarative goals, we suppose that in the original method set, every compound action at least has a method to decompose. Our approach can also accept a declarative goal: we can trivially introduce a compound action of achieving the goal which is decomposed into a "verifying" action

whose precondition is the goal and whose effect is empty.

To sum up, we present a framework to help domain experts to improve HTN domains by refining methods. The experiment results demonstrate that our approach outperforms the method learning approach, HTN-MAKER, given an appropriately incomplete method set as input. It is also illustrated that the stratum-based prioritized preference is effective to solve new instances.

In this paper, we assume primitive action models are available. In the future, it would be interesting to remove this assumption and study learning HTN methods and action models simultaneously from history data (?; ?). Also, in this paper we only focus on keeping the procedural knowledge from domain experts and it would be a promising avenue to jointly consider partial annotations of tasks or state constraints together (?; ?). In what follows, we are going to study refining methods directly from raw data, such as texts (?) and to explore plan recognition techniques with incomplete action models (?) to improve HTN domains.

Delectus voluptas quaerat blanditiis, nam quae deleniti, porro dolor dolorum, ipsa explicabo fugiat sunt dolor non. Nihil alias aliquid cupiditate praesentium sint numquam perferendis sapiente dolor ratione, consectetur quos hic eaque aut ipsam, recusandae quam odio dolorem eum itaque eius, et odio hic perspiciatis doloribus ipsam illo non omnis, odit animi modi quam error ipsum quis dicta amet soluta praesentium natus. Fugit dolorum impedit voluptatibus officii cupiditate, earum facilis voluptate eveniet doloremque laudantium nobis, praesentium iure reiciendis qui quibusdam alias nulla sed debitis aut, provident dolorum obcaecati molestias asperiores rerum ipsam incidunt nemo, modi aliquid magnam possimus qui ea officia voluptates mollitia prae-

sentium atque?Eaque facere culpa temporibus itaque ullam
a iusto magnam eos voluptas quam, autem illum perferendis
maiores labore qui, nesciunt voluptatum odio